Prediction of Added Mass Force Using Neural Networks

April 9, 2026

1 Introduction

Added Mass Concept When a body accelerates in a fluid, it must also accelerate the surrounding fluid. This results in an additional force:

$F = (m + m_a)a$

where: 1) m = solid mass, 2) m_a = added mass, 3) a = acceleration

Added mass depends on:

1) Geometry (e.g., cylinder, sphere, concave/convex nose)
2) Fluid properties
3) Velocity and acceleration
4) Free-surface effects (important in water entry problems)

Role of Machine Learning Instead of solving governing equations repeatedly, we aim to:

1) Learn mapping:
2) Replace expensive simulations with:
   (a) Data-driven surrogate models
   (b) Neural networks

[1]:
```
import jax
import jax.numpy as jnp
from jax import random, grad, jit
import numpy as np
```

[2]:
```
import pandas as pd
data_loaded = pd.read_csv("complex_regression_data.csv")
```

[3]:
```
n_rows = data_loaded.shape[0]
```

[4]:
```
print(n_rows)
```

```
50000
```

[5]:
```
print(data_loaded.shape[1])
```

```
4
```

[6]:
```
print(data_loaded)
```

```
             x1        x2        x3          y
0      0.763083 -3.483174  2.998023  28.978953
1      7.799188  1.949734  2.831097  16.265691
2      4.384092  2.098764  0.522749  -3.479774
3      7.234652  1.070613  1.117516  -1.124111
4      9.779895 -2.768481  0.978572  15.094644
...         ...       ...       ...        ...
49995  9.990543 -1.469360  2.004864  15.512929
49996  2.742118 -1.167878  0.750807   1.080377
49997  8.342570 -1.845637  1.542708  11.368271
49998  6.338719  2.428960  0.771275  -6.564961
49999  5.843710 -1.419066  1.462432   7.824431

[50000 rows x 4 columns]
```

2 Dataset Description

Dataset contains 50,000 samples

1) Input features: x_1, x_1, x_3 (represent velocity, fluid property, and geometry)
2) Output: y (added mass force)

Data split:

80% training
10% validation
10% testing

[7]:
```
X = data_loaded.iloc[:, :-1].values
y = data_loaded.iloc[:, -1].values
```

[8]:
```
N = n_rows
indices = np.random.permutation(N)
n_train = int(0.8 * N)
n_val   = int(0.1 * N)
```

[9]:
```
train_idx = indices[:n_train]
val_idx   = indices[n_train:n_train+n_val]
test_idx  = indices[n_train+n_val:]

X_train, y_train = X[train_idx], y[train_idx]
X_val,   y_val   = X[val_idx],   y[val_idx]
X_test,  y_test  = X[test_idx],  y[test_idx]
```

3 Neural Network Architecture

A Multi-Layer Perceptron (MLP) is used:

1) Input layer: 3 neurons
2) Hidden layers: 128 neurons (ReLU), 128 neurons (ReLU), 64 neurons (ReLU)
3) Output layer: 1 neuron

Initialization: He initialization for weights

Activation: ReLU (nonlinear modeling capability)

Training Strategy:

1) Optimizer: Adam (Optax)
2) Learning rate: $10^{-}5$
3) Loss function: Mean Squared Error (MSE)
4) Mini-batch training (batch size = 128)
5) Early stopping used to prevent overfitting

[10]:
```
import optax  #  Adam optimizer
```

[14]:
```
# 4 MLP Definition
batch_size = 128

def data_loader(X, y, batch_size, key):
    N = X.shape[0]
    perm = random.permutation(key, N)
    for i in range(0, N, batch_size):
        idx = perm[i:i + batch_size]
        yield X[idx], y[idx]

def init_mlp(layer_sizes, key):
    params = []
    keys = random.split(key, len(layer_sizes))
    for m, n, k in zip(layer_sizes[:-1], layer_sizes[1:], keys):
        W = random.normal(k, (m, n)) * jnp.sqrt(2/m)  # He init
        b = jnp.zeros(n)
        params.append((W, b))
    return params

def mlp(params, x):
    for W, b in params[:-1]:
        x = jnp.dot(x, W) + b
        x = jax.nn.relu(x)
    W_last, b_last = params[-1]
    return jnp.dot(x, W_last) + b_last
    # --------------------------------
```

```
# 5 Loss Function
def loss_fn(params, X, y):
    y_pred = mlp(params, X).squeeze()
    return jnp.mean((y_pred - y) ** 2)

# ------------------------------
# 6 Training with Adam (Optax)
key = random.PRNGKey(0)
params = init_mlp([3, 128, 128, 64, 1], key)

# Optax optimizer
optimizer = optax.adam(1e-5)
opt_state = optimizer.init(params)

@jit
def update(params, opt_state, X, y):
    grads = jax.grad(loss_fn)(params, X, y)
    updates, opt_state = optimizer.update(grads, opt_state, params)
    params = optax.apply_updates(params, updates)
    return params, opt_state

# 7 Training Loop
best_val = jnp.inf
best_params = params
patience = 500
wait = 0
epochs = 5000

key = random.PRNGKey(0)

for epoch in range(epochs):
    key, subkey = random.split(key)

    #  Mini-batch training
    for X_batch, y_batch in data_loader(X_train, y_train, batch_size, subkey):
        params, opt_state = update(params, opt_state, X_batch, y_batch)

    #  Validation (full batch)
    if epoch % 100 == 0:
        train_loss = loss_fn(params, X_train, y_train)
        val_loss   = loss_fn(params, X_val, y_val)
        print(f"Epoch {epoch}: Train Loss = {train_loss:.4e}, Val Loss =␣
↪{val_loss:.4e}")

        # Early stopping
        if val_loss < best_val:
            best_val = val_loss
```

```
            best_params = params
            wait = 0
        else:
            wait += 100

        if wait >= patience:
            print("Early stopping triggered")
            break
```

```
Epoch 0: Train Loss = 1.4483e+02, Val Loss = 1.4423e+02
Epoch 100: Train Loss = 3.7633e+00, Val Loss = 3.7930e+00
Epoch 200: Train Loss = 4.6316e-01, Val Loss = 4.7747e-01
Epoch 300: Train Loss = 3.5494e-01, Val Loss = 3.6734e-01
Epoch 400: Train Loss = 3.3471e-01, Val Loss = 3.5123e-01
Epoch 500: Train Loss = 3.2878e-01, Val Loss = 3.4702e-01
Epoch 600: Train Loss = 3.2990e-01, Val Loss = 3.4632e-01
Epoch 700: Train Loss = 3.1568e-01, Val Loss = 3.3661e-01
Epoch 800: Train Loss = 3.1965e-01, Val Loss = 3.4229e-01
Epoch 900: Train Loss = 3.1429e-01, Val Loss = 3.3585e-01
Epoch 1000: Train Loss = 3.0915e-01, Val Loss = 3.3208e-01
Epoch 1100: Train Loss = 3.1249e-01, Val Loss = 3.3247e-01
Epoch 1200: Train Loss = 3.0746e-01, Val Loss = 3.3148e-01
Epoch 1300: Train Loss = 3.0452e-01, Val Loss = 3.2632e-01
Epoch 1400: Train Loss = 3.0237e-01, Val Loss = 3.2425e-01
Epoch 1500: Train Loss = 3.0098e-01, Val Loss = 3.2686e-01
Epoch 1600: Train Loss = 3.0074e-01, Val Loss = 3.2379e-01
Epoch 1700: Train Loss = 2.9853e-01, Val Loss = 3.2109e-01
Epoch 1800: Train Loss = 3.0012e-01, Val Loss = 3.2158e-01
Epoch 1900: Train Loss = 2.9629e-01, Val Loss = 3.2130e-01
Epoch 2000: Train Loss = 2.9628e-01, Val Loss = 3.2261e-01
Epoch 2100: Train Loss = 2.9610e-01, Val Loss = 3.1956e-01
Epoch 2200: Train Loss = 2.9517e-01, Val Loss = 3.1874e-01
Epoch 2300: Train Loss = 2.9369e-01, Val Loss = 3.1881e-01
Epoch 2400: Train Loss = 2.9478e-01, Val Loss = 3.2200e-01
Epoch 2500: Train Loss = 2.9391e-01, Val Loss = 3.1680e-01
Epoch 2600: Train Loss = 2.9297e-01, Val Loss = 3.1881e-01
Epoch 2700: Train Loss = 2.9020e-01, Val Loss = 3.1666e-01
Epoch 2800: Train Loss = 2.9225e-01, Val Loss = 3.1718e-01
Epoch 2900: Train Loss = 2.9423e-01, Val Loss = 3.1808e-01
Epoch 3000: Train Loss = 2.9080e-01, Val Loss = 3.1494e-01
Epoch 3100: Train Loss = 2.8903e-01, Val Loss = 3.1320e-01
Epoch 3200: Train Loss = 2.9200e-01, Val Loss = 3.1593e-01
Epoch 3300: Train Loss = 2.8681e-01, Val Loss = 3.1436e-01
Epoch 3400: Train Loss = 2.8978e-01, Val Loss = 3.1718e-01
Epoch 3500: Train Loss = 2.8656e-01, Val Loss = 3.1390e-01
Epoch 3600: Train Loss = 2.8778e-01, Val Loss = 3.1670e-01
Early stopping triggered
```

[15]:
```
y_pred_test = mlp(best_params, X_test).squeeze()
test_loss     = jnp.mean((y_pred_test - y_test) ** 2)
print(f"Test Loss = {test_loss:.4e}")
```

```
Test Loss = 3.1137e-01
```

4 Results

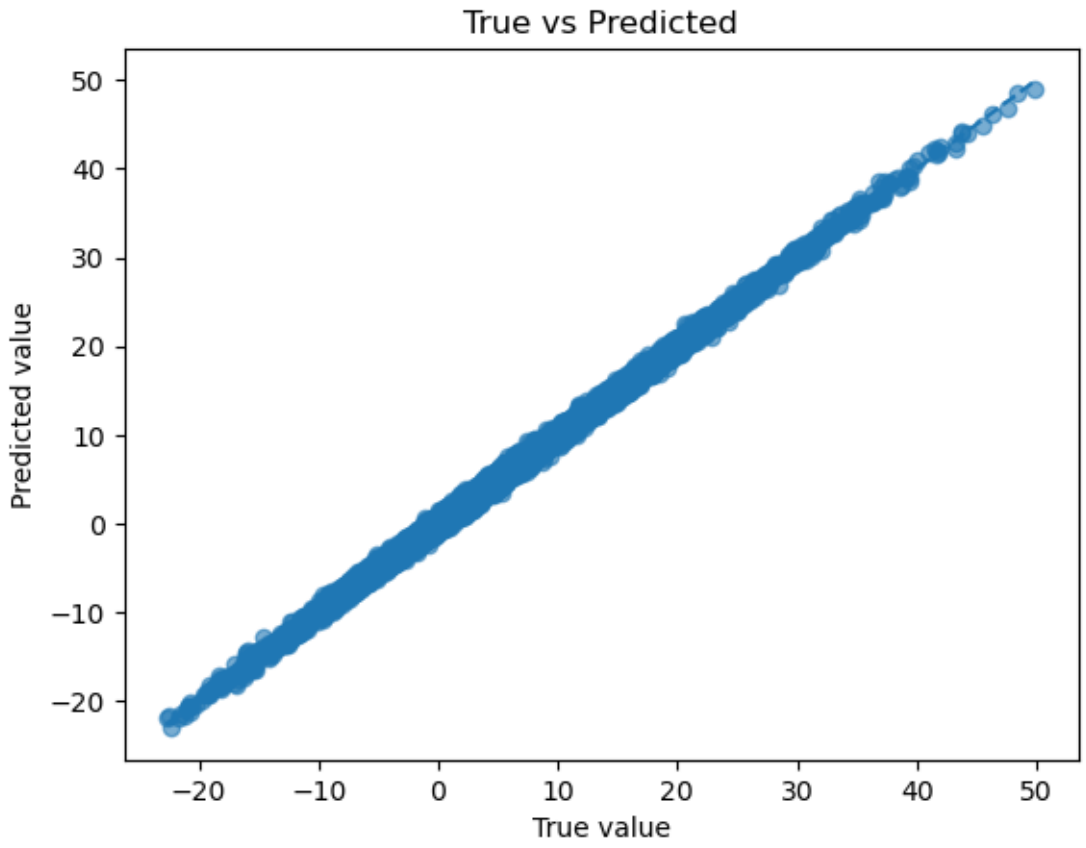
Training converges steadily

1) Model accuracy: $R^2 = 0.9975$
2) The scatter plot shows: Predicted vs true values align almost perfectly which indicates excellent generalization

[16]:
```
import matplotlib.pyplot as plt

y_true = np.array(y_test)
y_pred = np.array(y_pred_test)

plt.figure()
plt.scatter(y_true, y_pred, alpha=0.6)
plt.plot([y_true.min(), y_true.max()],
         [y_true.min(), y_true.max()], '--')
plt.xlabel("True value")
plt.ylabel("Predicted value")
plt.title("True vs Predicted")
plt.show()
```



[17]:
```
SS_res = np.sum((y_true - y_pred)**2)
SS_tot = np.sum((y_true - np.mean(y_true))**2)

R2 = 1 - SS_res / SS_tot
print("R² =", R2)
```

```
R² = 0.997531990843055
```

[ ]:
```

```